

Федеральное государственное автономное образовательное учреждение
высшего образования
“Национальный исследовательский университет ИТМО”

Дисциплина: Облачные и туманные вычисления
Лабораторная работа №1: Задача масштабирования (Docker -> Swarm)

Работу выполнил:
Рудкевич Илья Александрович
студент группы Р3406

Ход работы

Разработка основного приложения

Исходный код приложения: [ССЫЛКА](#)

Приложение возвращает кол-во запросов, сделанный к сервису:

```
~/dev/personal/itmo/cloud-and-fog-computing/lab1
● > curl localhost:5555
{"count":1}

~/dev/personal/itmo/cloud-and-fog-computing/lab1
● > curl localhost:5555
{"count":2}

~/dev/personal/itmo/cloud-and-fog-computing/lab1
● > curl localhost:5555
{"count":3}

~/dev/personal/itmo/cloud-and-fog-computing/lab1
● > curl localhost:5555
{"count":4}

~/dev/personal/itmo/cloud-and-fog-computing/lab1
● > curl localhost:5555
{"count":5}
```

Для запуска был написан Dockerfile, который собирает наше приложение:

dockerfile

```
9 FROM python:3.9-slim
8
7 ENV PYTHONDONTWRITEBYTECODE=1 \
6     PYTHONUNBUFFERED=1 \
5     FLASK_APP=main.py \
4     FLASK_RUN_HOST=0.0.0.0 \
3     FLASK_RUN_PORT=5555
2
1 WORKDIR /api
10
1 COPY api/requirements.txt ./
2 RUN pip install --no-cache-dir -r requirements.txt
3
4 COPY api/ ./
5
6 EXPOSE 5555
7
8 CMD ["flask", "run", "--host=0.0.0.0", "--port=5555"]
```

А также docker-compose, позволяющий запускать сервис в связке в Redis:

```
← →  makefile  dockerfile  • docker-compose.yml  .gitignore

docker-compose.yml

29  services:
28    app:
27      build: .
26      image: lab1-app:latest
25      ports:
24        - "5555:5555"
23      env_file:
22        - .env
21      environment:
20        - API_HOST=${API_HOST:-0.0.0.0}
19        - API_PORT=${API_PORT:-5555}
18        - REDIS_HOST=${REDIS_HOST:-redis}
17        - REDIS_PORT=${REDIS_PORT:-6379}
16        - REDIS_DB=${REDIS_DB:-0}
15        - REDIS_PASSWORD=${REDIS_PASSWORD}
14      depends_on:
13        - redis
12
11    redis:
10      image: redis:7-alpine
9      env_file:
8        - .env
7      command: ["redis-server", "--appendonly", "yes", "--requirepass", "${REDIS_PASSWORD}"]
6      volumes:
5        - redis-data:/data
4      deploy:
3        replicas: 1
2        restart_policy:
1          condition: any
30
1  volumes:
2    redis-data:
```

Инициализация Swarm и развертывание стека

Для начала инициализируем docker swarm:

```
7 dev, personal, home, cloud and fog computing, etc.
• > docker swarm init
Swarm initialized: current node (32wtalx3r6wltclonlaa8gq3n) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-28dxhwh6alj7yopossx5byocxr0ncupyqq7hn8t3nuj6hkduxg-0uwbtejuq0iq3e736t6mmohv 192.168.104.1:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

И развернем стек:

```
• > make deploy
docker compose build app
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[+] Building 0.4s (11/11) FINISHED
=> [app internal] load build definition from dockerfile
=> => transferring dockerfile: 408B
=> [app internal] load metadata for docker.io/library/python:3.9-slim
=> [app internal] load .dockerignore
=> => transferring context: 2B
=> [app 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> [app internal] load build context
=> => transferring context: 128.78kB
=> CACHED [app 2/5] WORKDIR /api
=> CACHED [app 3/5] COPY api/requirements.txt ./
=> CACHED [app 4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [app 5/5] COPY api/ ./
=> [app] exporting to image
=> => exporting layers
=> => writing image sha256:1bfbf87dcca4719df5300862468d1d6c6725d355bb9b1e14947564c5f79dd47
=> => naming to docker.io/library/lab1-app:latest
=> [app] resolving provenance for metadata file
[+] Building 1/1
✔ app
docker stack deploy -c docker-compose.yml lab1
Ignoring unsupported options: build

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network lab1_default
Creating service lab1_app
Creating service lab1_redis

• > make status
docker stack ps lab1
ID                NAME                IMAGE                NODE                DESIRED STATE        CURRENT STATE        ERROR                PORTS
dij1r56y7qnn     lab1_app.1          lab1-app:latest      lima-avito          Running               Running 7 minutes ago
gir55k7jxhz1     lab1_redis.1        redis:7-alpine       lima-avito          Running               Running 7 minutes ago
docker stack services lab1
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
pxf6oot2z98c     lab1_app            replicated          1/1                 lab1-app:latest     *:5555->5555/tcp
l7yqvus0kpvx     lab1_redis          replicated          1/1                 redis:7-alpine
```

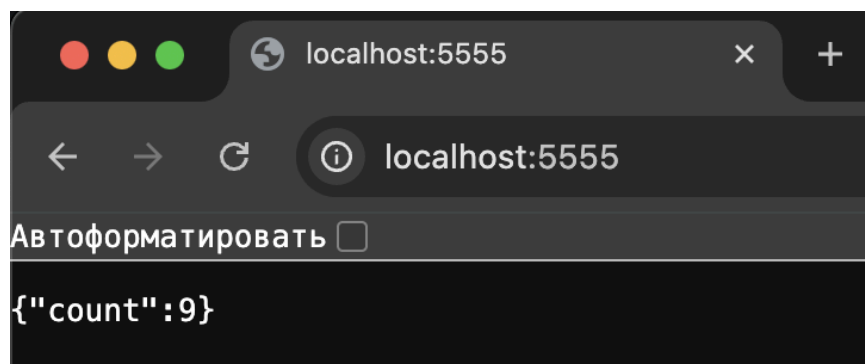
Тестирование на отказ

```
lab1 git:master*
> make status
docker stack ps lab1
ID                NAME                IMAGE                NODE                DESIRED STATE        CURRENT STATE        ERROR                PORTS
t58bzpxoyv4a     lab1_app.1          lab1-app:latest      lima-avito          Running               Running 4 minutes ago
muvs4p8hc30z     lab1_redis.1        redis:7-alpine       lima-avito          Running               Running 4 minutes ago
docker stack services lab1
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
l6e3n3ehntn5     lab1_app            replicated          1/1                 lab1-app:latest     *:5555->5555/tcp
xaq8o5bktnf8     lab1_redis          replicated          1/1                 redis:7-alpine

lab1 git:master*
> docker kill lab1_app.1.t58bzpxoyv4atf4siomhh1u6h
lab1_app.1.t58bzpxoyv4atf4siomhh1u6h

lab1 git:master*
> make status
docker stack ps lab1
ID                NAME                IMAGE                NODE                DESIRED STATE        CURRENT STATE        ERROR                PORTS
t07pl5e7dgrn     lab1_app.1          lab1-app:latest      lima-avito          Running               Running 2 seconds ago
t58bzpxoyv4a     \_ lab1_app.1       lab1-app:latest      lima-avito          Shutdown              Failed 7 seconds ago    "task: non-zero exit (137)"
muvs4p8hc30z     lab1_redis.1        redis:7-alpine       lima-avito          Running               Running 4 minutes ago
docker stack services lab1
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
l6e3n3ehntn5     lab1_app            replicated          1/1                 lab1-app:latest     *:5555->5555/tcp
xaq8o5bktnf8     lab1_redis          replicated          1/1                 redis:7-alpine
```

Сервис продолжил работу



Масштабирование

Запустим основной сервис в 5 репликах:

```
> make scale
docker service scale lab1_app=5
lab1_app scaled to 5
overall progress: 5 out of 5 tasks
1/5: running [=====>]
2/5: running [=====>]
3/5: running [=====>]
4/5: running [=====>]
5/5: running [=====>]
verify: Service lab1_app converged
docker stack ps lab1
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
t07pl5e7dgrn	lab1_app.1	lab1-app:latest	lima-avito	Running	Running 17 minutes ago	
t58bzpoxyv4a	_ lab1_app.1	lab1-app:latest	lima-avito	Shutdown	Failed 17 minutes ago	"task: non-zero exit (137)"
vze128wmqbur	lab1_app.2	lab1-app:latest	lima-avito	Running	Running 5 seconds ago	
m24bd4dqs02	lab1_app.3	lab1-app:latest	lima-avito	Running	Running 5 seconds ago	
detex3hd46zp	lab1_app.4	lab1-app:latest	lima-avito	Running	Running 5 seconds ago	
xemq06zyledo	lab1_app.5	lab1-app:latest	lima-avito	Running	Running 5 seconds ago	
muvs4p8hc30z	lab1_redis.1	redis:7-alpine	lima-avito	Running	Running 21 minutes ago	

```
docker stack services lab1
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
l6e3n3ehntn5	lab1_app	replicated	5/5	lab1-app:latest	*:5555->5555/tcp
xaq8o5bktnf8	lab1_redis	replicated	1/1	redis:7-alpine	

Заключение

В ходе лабораторной работы создано и развернуто контейнеризованное веб-приложение на Flask с базой данных в Redis. Изначально проект был настроен для локальной разработки с использованием Docker Compose, после чего та же конфигурация успешно перенесена на кластер Docker Swarm. В процессе демонстрируются основные возможности оркестратора: устойчивость к сбоям (автоматическое восстановление сервиса) и горизонтальное масштабирование (увеличение числа экземпляров сервиса Flask до пяти).